

1. Class-Level Annotations & Setup

These annotations tell Spring how to treat this class and wire up its dependencies.

- `@Configuration`: Marks this class as a source of bean definitions for the Spring application context.
- `@EnableWebSecurity`: Switches off the default Spring Boot security auto-configuration and tells Spring to strictly follow the rules defined in this class.
- `@RequiredArgsConstructor`: A Lombok annotation that automatically generates a constructor for all final fields. This is how Spring injects your custom filters and handlers (like `JwtAuthenticationFilter` and `AuthenticationProvider`) into this class—known as Constructor Injection.
- `@EnableMethodSecurity`: Enables method-level security annotations like `@PreAuthorize("hasRole('ADMIN')")` so you can secure individual service or controller methods, not just URLs.

2. Environment Variables (@Value)

You are injecting several variables directly from your `application.properties` or `application.yml` file:

- `scim.token`: A static token used for SCIM (System for Cross-domain Identity Management) provisioning.
- `allowedOrigins`, `allowCredentials`, `maxAge`: Configuration parameters for Cross-Origin Resource Sharing (CORS).
- `basePath`: The servlet context path, allowing your endpoint matchers to dynamically adjust if the app's base URL changes.

3. The Core: `securityFilterChain(HttpSecurity http)`

This method builds the `SecurityFilterChain`, which is a list of filters that every incoming HTTP request must pass through.

A. CSRF and CORS

- `.csrf(csrf -> csrf.disable())`: Disables Cross-Site Request Forgery protection. CSRF protection requires a server-side session to store a CSRF token. Since your API is stateless and uses tokens (like JWTs) in the HTTP headers, CSRF attacks are neutralized by design. Disabling it is standard practice for REST APIs.
- `.cors(cors -> cors.configurationSource(corsConfigurationSource()))`: Activates CORS and points it to the custom configuration defined later in the class.

B. Security Headers (Defense in Depth)

Your configuration includes an exceptionally strict set of HTTP response headers to protect the browser from executing malicious payloads.

- **Frame Options (deny):** Prevents your site from being embedded in an <iframe> on another site, neutralizing "Clickjacking" attacks.
- **XSS Protection (disable):** You explicitly disable the browser's legacy XSS filter. This is actually correct modern practice because legacy XSS filters often introduced *new* vulnerabilities. Modern apps rely on CSP instead.
- **Cache Control (disable & StaticHeadersWriter):** Prevents the browser from caching sensitive API responses (no-store, no-cache).
- **X-Content-Type-Options (nosniff):** Forces the browser to strictly follow the declared Content-Type. If an attacker uploads a malicious .html file disguised as a .jpg, the browser will refuse to execute it as HTML.
- **HSTS (httpStrictTransportSecurity):** Tells the browser, "For the next year (31,536,000 seconds), only ever communicate with this site over HTTPS, even if the user types HTTP." includeSubDomains(true) applies this to all subdomains.
- **Content Security Policy (CSP):** An incredibly strict policy.
 - default-src 'self': Only load resources from your own domain.
 - object-src 'none': Blocks plugins like Flash or Java applets.
 - upgrade-insecure-requests: Forces HTTP links to upgrade to HTTPS.
- **Referrer Policy (NO_REFERRER):** Ensures that when a user clicks a link taking them away from your app, your app's URL (which might contain sensitive query parameters) is not sent to the destination site.
- **Permissions-Policy:** Restricts access to browser features. Your app allows geolocation only for itself (self) and completely disables microphone access (()).
- **Cross-Origin-Embedder-Policy (require-corp):** Prevents your site from loading resources from other domains unless those domains explicitly grant permission.

C. Authorization Rules (authorizeHttpRequests)

- **Custom Rule:**
.requestMatchers("/sc/").access(buildDomainAccessAuthorizationManager())
delegates the authorization decision for any /sc/ endpoint to your custom domain checker.
- **Public Endpoints (permitAll):** You are opening up paths necessary for authentication (/auth/), automated webhooks (/scim/, /audio/, /twiml/),

WebSockets (/ws-message/), and Swagger/OpenAPI documentation (/v3/api-docs, /swagger-ui/).

- **Catch-All:** `.anyRequest().authenticated()` dictates that if an endpoint isn't explicitly listed above, the user must be authenticated.

D. Session Management & Filters

- `.sessionManagement(...)`: Set to `STATELESS`. Spring Security will not create an `HttpSession`. It won't remember the user between requests; every single request must carry its own authentication (like a JWT).
- `.addFilterBefore(...)`: Spring Security defaults to checking for a username and password (`UsernamePasswordAuthenticationFilter`). Because you are using stateless tokens, you are injecting custom filters to run *before* that step:
 1. `jwtAuthFilter`: Intercepts the request to validate a JWT.
 2. `actionableMessageAuthenticationFilter`: Intercepts specific actionable message requests.
 3. `ScimAuthFilter`: Validates the static `scimToken` for SCIM endpoints.

E. Logout Configuration

- `.logoutUrl("/auth/logout")`: Defines the endpoint that triggers the logout process.
- `.addLogoutHandler(logoutHandler)`: Executes your custom logic (e.g., blacklisting a JWT or deleting a refresh token from the database).
- `.logoutSuccessHandler(...)`: Clears the `SecurityContextHolder` so the application completely forgets the user's current security context.

4. CORS Definition: `corsConfigurationSource()`

This method tells the browser which external domains are allowed to communicate with your API.

- **setAllowedOrigins**: Reads your comma-separated list of allowed frontend URLs (e.g., `https://myapp.com`).
- **setAllowedMethods**: Explicitly lists allowed HTTP verbs.
- **setAllowedHeaders**: Dictates which headers the frontend is allowed to send (e.g., `Authorization`).
- **setAllowCredentials**: If true, allows the frontend to send credentials (like cookies or HTTP auth) across domains.

- **setMaxAge:** Tells the browser to cache this CORS "pre-flight" check for a certain amount of time, reducing latency on subsequent requests.
- **registerCorsConfiguration("/", configuration):** Applies these rules to all API endpoints.

5. Custom Domain Authorization: buildDomainAccessAuthorizationManager()

This is the custom logic applied specifically to the /sc/ endpoints.

- It grabs the allowed origins and splits them into a list.
- It creates a lambda function that inspects the incoming Host header of the HTTP request.
- If the Host header contains one of the allowed domains, it returns new AuthorizationDecision(true) (Access Granted).
- If it does not match, it returns false (Access Denied - 403 Forbidden).
- *Note on usage:* This is often used for multi-tenant applications or for securing webhook endpoints where you know exactly which host should be routing the traffic.